If you must use a floating point variable as a loop index, try to use fractions that are a power of *two* (such as: 1/2, 1/4, 3/8, 27/16), instead of a power of *ten* (such as: 0.1, 0.6, 1.4, 2.3, etc.). For instance, it would be better to use: FOR X = 1 TO 10 STEP 0.125, rather than: FOR X = 1 to 10 STEP 0.1. This allows the index to always have an *exact* binary representation, thereby reducing round-off error. For example, the decimal number: 1.125, can be represented exactly in binary notation: $1.001000000000000000000000 \times 2^0$. In comparison, the decimal number: 1.1, falls *between* two floating point numbers: 1.0999999046 and 1.1000000238 (in binary these numbers are: $1.00011001100110011001100 \times 2^0$ and $1.00011001100110011001101 \times 2^0$). This results in an inherent error each time 1.1 is encountered in a program.

A useful fact to remember: single precision floating point has an *exact* binary representation for every whole number between ±16.8 million (to be exact, $\pm 2^{24}$). Above this value, the gaps between the levels are larger than one, causing some whole number values to be missed. This allows floating point whole numbers (between ±16.8 million) to be added, subtracted and multiplied, with no round-off error.

Execution Speed: Program Language

DSP programming can be loosely divided into three levels of sophistication: *Assembly*, *Compiled*, and *Application Specific*. To understand the difference between these three, we need to start with the very basics of digital electronics. All microprocessors are based around a set of internal binary registers, that is, a group of flip-flops that can store a series of ones and zeros. For example, the 8088 microprocessor, the core of the original IBM PC, has *four* general purpose registers, each consisting of 16 bits. These are identified by the names: AX, BX, CX, and DX. There are also *nine* additional registers with special purposes, called: SI, DI, SP, BP, CS, DS, SS, ES, and IP. For example, IP, the Instruction Pointer, keeps track of where in memory the next instruction resides.

Suppose you write a program to add the numbers: 1234 and 4321. When the program begins, IP contains the address of a section of memory that contains a pattern of ones and zeros, as shown in Table 4-3. Although it looks meaningless to most humans, this pattern of ones and zeros contains all of the commands and data required to complete the task. For example, when the microprocessor encounters the bit pattern: 00000011 11000011, it interpreters it as a command to take the 16 bits stored in the BX register, add them in binary to the 16 bits stored in the AX register, and store the result in the AX register. This level of programming is called **machine code**, and is only a hair above working with the actual electronic circuits.

Since working in binary will eventually drive even the most patient engineer crazy, these patterns of ones and zeros are assigned names according to the function they perform. This level of programming is called **assembly**, and an example is shown in Table 4-4. Although an assembly program is much easier to understand, it is fundamentally the same as programming in